Digital Systems S26 - Assembly Programming Project

2026.06.02

Important dates

- Team registration deadline: 2026-05-13 23:59
- Projects assigned on: 2026-05-15
- Send levels to teacher: Thursday, June 4th 23:59.
- Submission deadline: Monday, June 8th 23:59 (48h late allowed, -0.5/day)
- Presentations: Thursday 11th and Friday 12th

Project Overview

Each team must implement the mechanics of **one** of the following games in Game Boy assembly language:

- **Catrap**
- **Rock Slide**

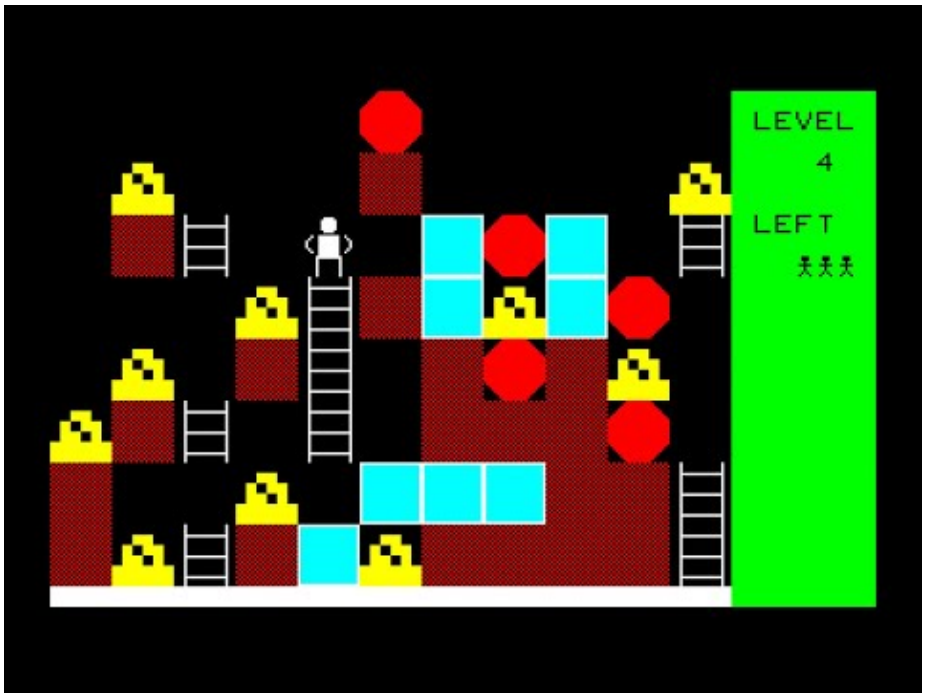
Both games are single-player, turn-based puzzle games with mechanics similar to *Sokoban* and *Boulder Dash*.

A key objective of this project is **reverse engineering** the original game mechanics. Students are expected to:

- Play and/or watch the original game
- Identify all tile types
- Analyze interactions and movement rules
- Recreate the gameplay behavior as accurately as possible

Understanding and documenting the original mechanics is considered part of the assignment.

Catrap



Catrap first appeared as **Pitman** in 1985 for the Sharp MZ-700 computer and was adapted to the Game Boy in 1990.

- Get the file `catrap.gb` from the "Original game files" folder on Moodle and run it on an emulator like BGB, Emulicious or Sameboy.
- Find a video gameplay of the original version in the same folder (`pitman.mp4`).

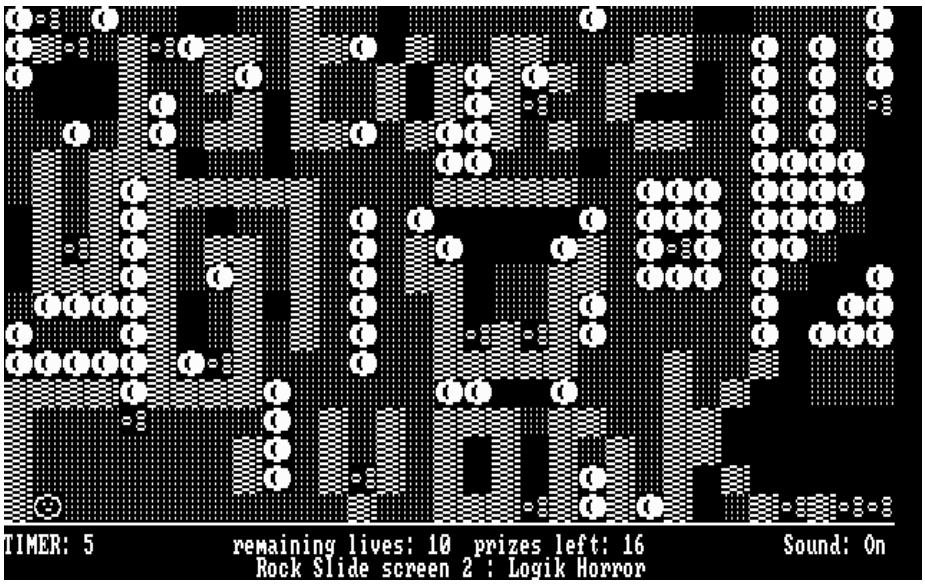
Do **NOT** implement:

- the rewind feature
- levels with 2 players
- timer

The following level tiles encoding must be followed:

```
0 = Nothing
1 = Stairs
2 = Monster
3 = Rock
4 = Sand
5 = Wall
6 = Ghost
7 = Player
```

Rock Slide



Rock Slide is a DOS game released in 1989 by Viking Tecnologies. This of it as Boulder Dash without enemies and simpler rocks fall mechanics. Gravity does not affect the player, but it does to rocks, and rocks can kill the player.

- Get the file `Rock-Slide_DOS_EN.zip` from the "Original game files" folder on Moodle and run it on the DOSBOX emulator (you will need to install DOSBOX on your system and learn to use it).
- You can also open `SLIDER.DAT` with a text editor to see the levels.
- Find a video gameplay in the same folder (`rockslide.mp4`).

The tiles:

```
0 = Dirt
1 = Rock
2 = NPC
3 = Money
4 = Solid wall
5 = Empty space
6 = Player start
```

Do implement:

- Live counter (levels start with 9 lives)

Do **not** implement:

- Random levels
- Timer

Assigned Teams

To Catrap:

- CatGirlHunter
- wwwww
- FlashSync
- Catrap123
- KingBob
- Butcher
- LeBang

To Rock Slide:

- hhhhh
- happy cat
- YSGG
- EasyBoys
- TriCircuit
- wwc
- XLYG

Common Features to Implement

All games must include the following sequence:

1. Title screen.
2. Start game from first level when any key is pressed
3. When a level is cleared, the next level starts.
4. When the last level is cleared, an ending screen is shown.
5. Return to the title screen after a key press.

Controls during the game:

- Player is controlled by the 4 direction keys
- Start resets the level
- `A` changes to the next level and `B` changes to the previous level

Features for teams of 3 (each feature not implemented yields a -1 to the grade):

- Step counter feature
- Smooth transitions in player and tiles movements

Levels

- Each team is required to send to the teacher one level per team member (in RGBASM source format), before June 4th 23:59. For each missing level the team may lose 1 point.

- These levels will be curated by the teacher and shared on Moodle for all teams to use (teams may choose to use them or not).
- For Catrap, a level is stored as a tilemap of 12 columns and 12 rows.
- For Rock Slide, a level is stored as a tilemap of 20 columns and 16 rows.
- The "useful" size of a level (where the player can move) may be smaller than the above sizes.
- Projects must contain at least 10 levels.

Checklist for both projects

Each unfulfilled item may bring your grade down by 1 point, except when specified:

- Use meaningful and descriptive names for functions, variables, and labels (check the meaning of English words you use).
- Use local labels to improve code structure.
- Simplify the control flow of your code where possible (check conditional jumps, calls and returns).
- Use consistent indentation and formatting.
- Remove unreachable code, unused labels, variables and data.
- Prefer smaller code in general (in number of instructions) unless readability is affected.
- Include comments explaining key sections of your code, especially complex algorithms or game logic, but do not comment *every* line.
- Where relevant, describe functions parameters and return values.
- Make sure your program has a clear separation between game logic and data.
- Do not use magic values that appear several times in the code; instead use named constants.
- The stack pointer must be at the same value after you finish all levels and are back at title screen.
- The report must contain the cycle calculations outside and inside of VBlank.
- The report must fulfil each requirement of the "submission requirements".
- Only access OAM and VRAM during VBlank when the PPU is on. When the game is ongoing, update screen at every frame (no stuttering). Do not turn off PPU mid-game. (2 points)

Project Grading

Projects are evaluated holistically.

Evaluation criteria include:

- Completion of required features
- Accuracy of gameplay mechanics
- Code quality and organization

- Technical correctness
- Report quality
- Team size and contribution

Grades are assigned individually.

Students who cannot explain the submitted project during the presentation may receive a lower grade.

Submission Requirements

1. **File Structure:**
   - `main.asm`: Contains all the project's code, data (including levels) and variables declarations.
     - Must include the standard `hardware.inc` header.
     - Must begin with a comment listing the names and IDs of all team members.
   - `report.pdf`: An A4 PDF document of **no more than 6 pages** with the following structure:
     1. Names and IDs of all team members + URLs and/or names of any existing resource reused in the project.
     2. Description of how to play the game. Keys. Features implemented.
     3. Description of the implementation. Relevant data structures and algorithms used.
     4. Shortcomings of the implementation, if any.
     5. How WRAM is used and how many bytes are used in total.
     6. Cycles calculations of VRAM/OAM update time during VBlank.
     7. Cycles calculations of CPU use outside of VBlank (optional).
   - Combined file sizes must not exceed 10 MB.
2. **Assembly Compatibility:**
   - The recommended way to develop and test your project is https://gbdev.io/rgbds-live/. It includes a debugger with breakpoints, step execution, and memory viewers. You can download the final `.gb` file from RGBDS-live for testing with a more precise emulator (BGB, Emulicious, Sameboy).
   - If you prefer to work locally, you may use `rgbasm`, `rgblink`, and `rgbfix` (version 1.0 or newer).
   - The submitted `main.asm` must assemble correctly on RGBDS-live for grading.
   - Submissions failing to meet this requirement will not be graded (0 points).

Restrictions

The following are prohibited:

1. Pre-existing game engines or frameworks
2. Features beyond course scope
3. DMA (Direct Memory Access)
4. Interrupts
5. `rst` instructions

Additional constraints:

6. ROM size must not exceed 32 KB
7. Use the same set of tiles for objects and background (`LCDC_BLOCK01`).
8. Input reading is done with the `readKeys` function provided in the course
9. Tiles are stored in ROM as 8 bytes each, and are be copied to VRAM by doubling each byte (palettes changes are allowed)

Presentations

After submission, a presentation happens in which all group members must be present.

In the projects presentations, the teacher may choose that only one of the students present the project, with no help from the others. Make sure everyone involved is actively participating to it and understands the submission.

Presentations are done on one of the student's computer; the teacher will also have the game running on his computer, and will ask questions and talk with you while you present your project.

The group may be asked to do a second presentation.

You can use the submitted report of your project to guide your presentation. Do not bring another report or presentation file.

After the Presentations

- The grades of all teams will be published together on Moodle; no additional feedback will be provided by defaul (write an e-mail if more is needed).
- After grades publications, project can be resubmitted to improve the grade by up to 2 more points, following a list of items proposed by the teacher.
- Final project grades will be set before Exam A.

Questions and Support

- You are welcome to ask questions about the project, but please make sure you have already tried something on your own or have a specific idea to

discuss.
- Vague questions like “How do I start?”, “What should I do?” are not appropriate if you have not attempted to understand or plan the task.

Start early to ensure you have time to solve issues and ask meaningful questions. The teacher will not be available from June 6th to 8th.

Academic Integrity

All work must be completed exclusively by team members, with any reused code or data explicitly cited in the `.asm` file (e.g., URLs or names). One exception is the code that is provided by the course. Cheating or misrepresentation will result in a zero for the project **and** all assignment grades. All students of the group take full responsibility for the files uploaded to Moodle on their behalf. Students involved remain eligible for the final exam.